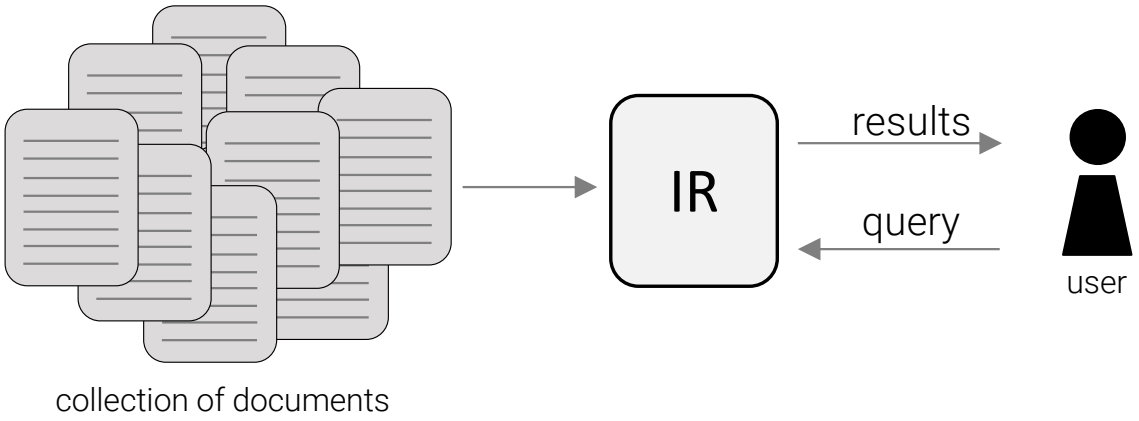
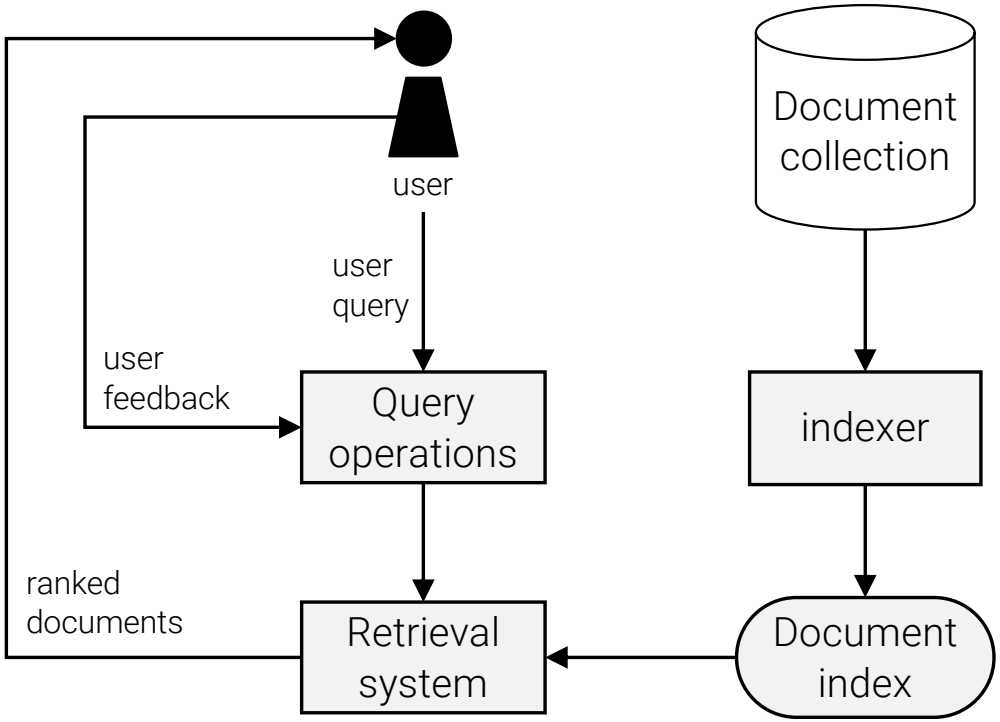
Module 3:
Information retrieval and web search

Information Retrieval

- Web search has roots in IR



IR general architecture



Information retrieval models

- IR model
  - how to represent documents and queries?
  - how to compute relevance of a document d to a query q?
- Main IR models:
  - Boolean model,
  - vector space model,
  - language model,
  - probabilistic model.

IR model formalization

- D – collection of documents,
- $V = \{t_1, t_2, \ldots, t_{|V|}\}$ – set of distinctive terms t_i in D – called vocabulary of D with size $|V|$,
- Each term t_i of a document $\boldsymbol{d}_j \in D$ has a weight $w_{ij} > 0$.
- Each document $\boldsymbol{d}_j$ is represented with a term vector: $\boldsymbol{d}_j = (w_{1j}, w_{2j}, \ldots, w_{|V|j})$, where w_{ij} corresponds to $t_i \in \mathrm{V}$ signifying the importance of t_i in $\boldsymbol{d}_j$.

Boolean IR model

- Simplest IR model, queries and documents represented as sets of terms (with vectors):

$q = \{t_i, tp, tj\}$, where $t_i, t_p, t_j \in V$

	t_1	t_2	…	t_i	…	t_p	…	t_j	…	$t_{\|V\|}$
q	0	0	…	1	…	1	…	1	…	0

- Retrieval:
  - Exact matching based on Boolean algebra (AND, OR, NOT)

`query:` `computer NOT science`

Vector space model...

- Best known and widely used in IR
- Documents represented as weight vectors
- Variations of TF/TF-IDF used for weight calculation
- TF – term frequency:
  - $w_{ij} = f_{ij}$ where f_{ij} number of times t_i appears in $\boldsymbol{d}_j$.
  - TF can be normalized, e.g. Euclidean normalization:

$$tf_{ij} = \frac{f_{ij}}{\sqrt{f_{1j}^2 + f_{1j}^2 + \cdots + f_{|V|j}^2}}$$

TF-IDF scheme

- TF: non discriminative words (e.g. “the”, “this”,...) can have high weights.
- TF-IDF scheme relativizes terms appearing in many documents.
- Computed e.g. as $idf_i = log \frac{N}{df_i}$
  - N total number of documents,
  - d_{fi} the number of documents with t_i.
- According to TF-IDF, $w_{ij} = tf_{ij} * idf_{ij}$

TF-IDF for queries

- Number of variations for calculating IDF exists.
- How about queries?
- Sulton and Buckley (1988)* suggests:

$$w_{ij} = \left(0.5 + \frac{0.5 * f_{ij}}{\max\{f_{1j}, f_{2j}, \ldots, f_{|V|j}\}}\right) * log \frac{N}{df_i}$$

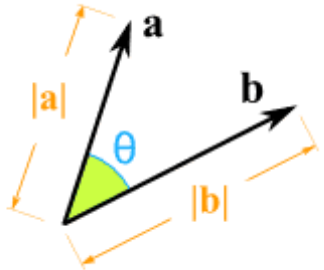
*Salton, G. and C. Buckley. *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, 1988, 24(5): p. 513-523.

Pivot normalization

- Describe and show with online graphs how pivot normliaztaion works

Relevance ranking in vector space model

- Documents relevance ranking:
  - how relevant is $\boldsymbol{d}_j$ to q?
  - many variants to measure relevance
- Dot product: $sim(\boldsymbol{d}_j, \boldsymbol{q}) = \langle \boldsymbol{d}_j \cdot \boldsymbol{q} \rangle$
- Cosine similarity:



$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta)$

$\mathbf{a} \cdot \mathbf{b} = a_x \times b_x + a_y \times b_y$

$$cosine(\boldsymbol{d}_j, \boldsymbol{q}) = \frac{\langle \boldsymbol{d}_j \cdot \boldsymbol{q} \rangle}{\|\boldsymbol{d}_j\| * \|\boldsymbol{q}\|} = \frac{\sum_{i=1}^{|V|} w_{ij} * w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} * \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

Okapi method

- Variation of a relevance measure, Okapi method*:

$$okapi(d_j, q) = \sum_{t_i \in q, d_j} ln \frac{N - df_i + 0.5}{df_i + 0.5} * \frac{(k_1+1) f_{ij}}{k_1\left(1 - b + b\frac{dl_j}{avdl}\right) + f_{ij}} * \frac{(k_2+1) f_{iq}}{k_2 + f_{iq}}$$

  - where:
    - df_i is the number of documents that contain the term t_i
    - dl_j is the document length (in bytes) of d_j
    - $avdl$ is the average document length of the collection
    - Parameters: k_1 (between 1.0-2.0), b (usually 0.75) and k_2 (between 1-1000).

*Robertson, S., S. Walker, and M. Beaulieu. *Okapi at TREC-7: automatic ad hoc filtering*, VLC and interactive track. NIST Special Publications, 1999: p. 253-264.

Statistical language model

- Based on probability, funded on statistical theory.
- Idea:
  - for each document d estimate a language model
  - rank documents by the likelihood of the query q given the language model.
- Similar approaches used in NLP and speech recognition.
- First proposed by Ponte and Croft*

*Ponte, J. and W. Croft. *A language modeling approach to information retrieval*. In Proceedings of ACM SIGIR Conf. on Research and Development in Information Retrieval (SIGIR-1998), 1998..

Formalization

- We are interested in estimating $\Pr(q|d_j)$, where
  - $q = q_1 q_2 \dots q_m$ is a query
  - D is document collection and $d_j \in D$
- Typically based on unigrams
- Better models consider n-grams

Multinominal distribution

- Multinominal distribution over unigrams (words):

$$Pr(q|d_j) = \prod_{i=1}^{m} \Pr(q_i|d_j) = \prod_{i=1}^{|V|} \Pr(t_i|d_j)^{f_{iq}}$$

  - where
    - $q = q_1 q_2 \dots q_m$,
    - V is vocabulary of d_j
    - $t_i \in V$ and f_{iq} tells the number of times t_i appears in q
    - $\Pr(t_i|d_j) = \frac{f_{ij}}{|d_j|}$ and $\sum_{i=1}^{|V|} \Pr(t_i|d_j) = 1$

Laplace and Lindstone smoothing

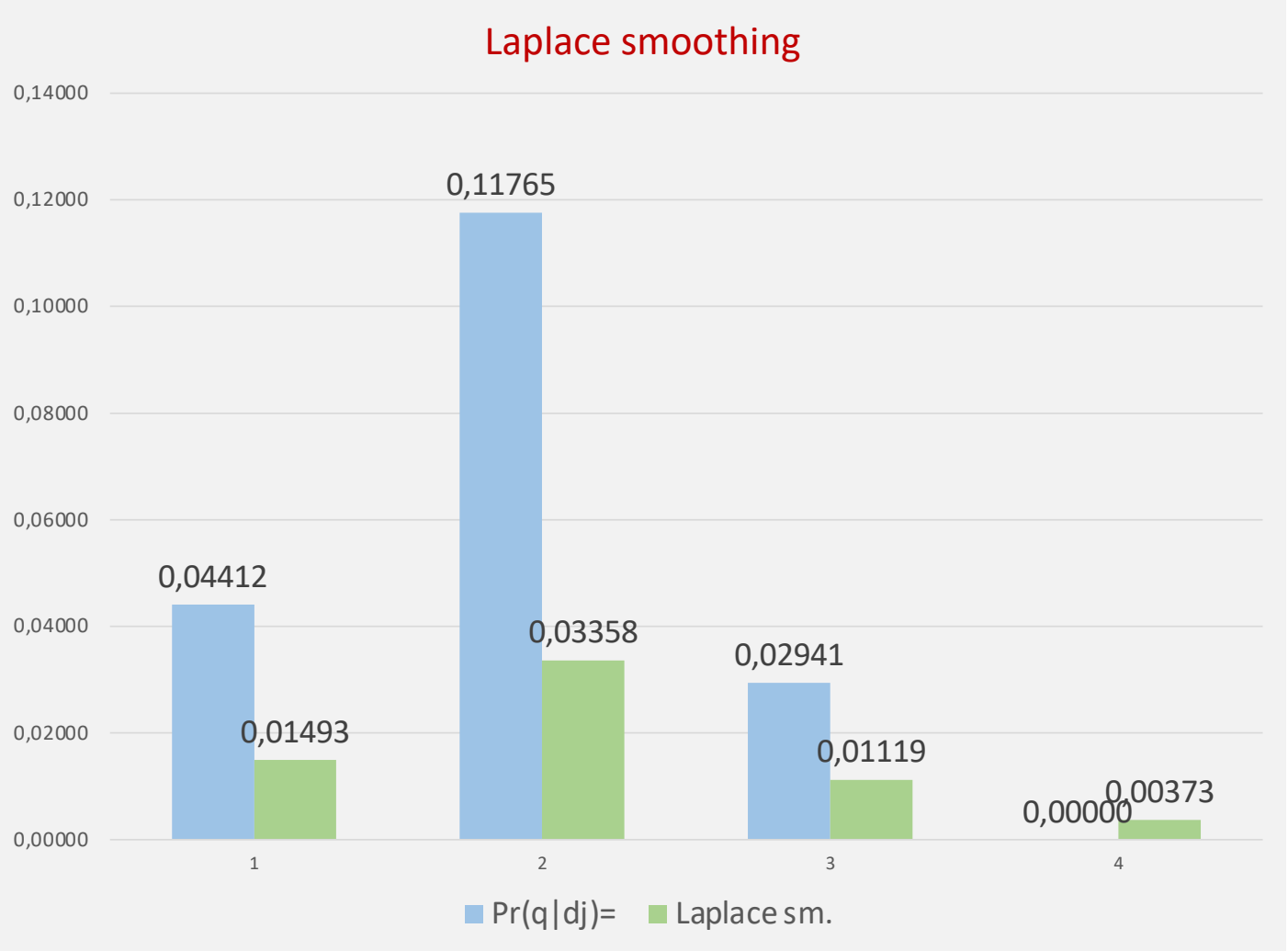
- What if $t_i \in q$ and $t_i \notin d_j \Longrightarrow \Pr(t_i|d_j) = 0$?
- Additive smoothing:

$$Pr_{add}(t_i|d_j) = \frac{\lambda + f_{ij}}{\lambda|V| + |d_j|}$$

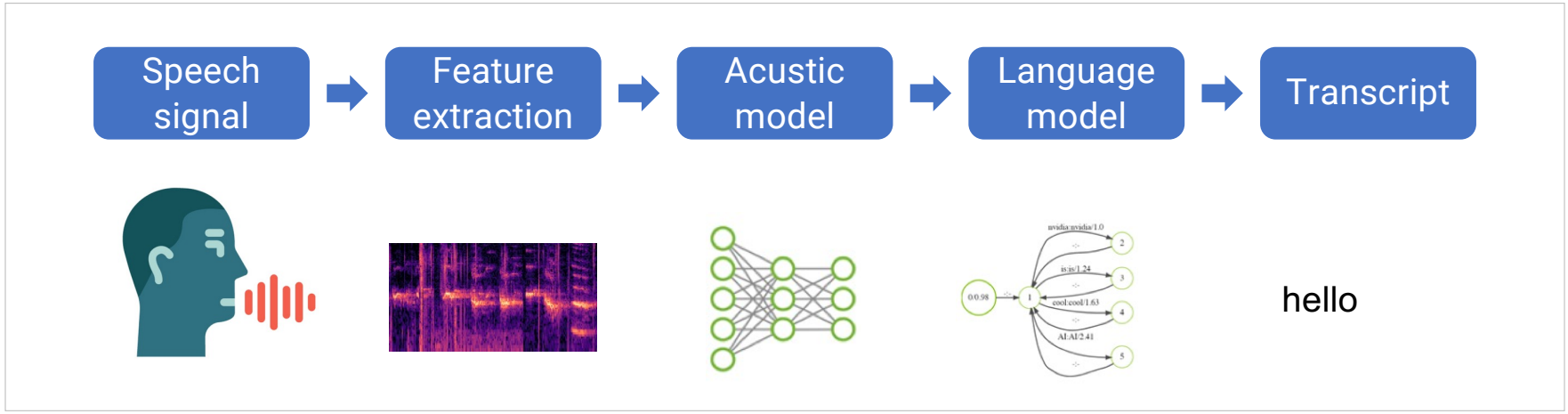
- Laplace smoothing: $\lambda = 1$
- Lindstone smoothing: $0 < \lambda < 1$

Example

- $q = t_1 t_2 t_3 t_4$
- $|V| = 200$
- $|d_j| = 68$
- $f_{1j} = 3$
- $f_{2j} = 8$
- $f_{3j} = 2$
- $f_{4j} = 0$



Speech recognition



word sequence: $W = w_1, w_2, \ldots, w_m$

acustic features: $X = x_1, x_2, \ldots, w_n$

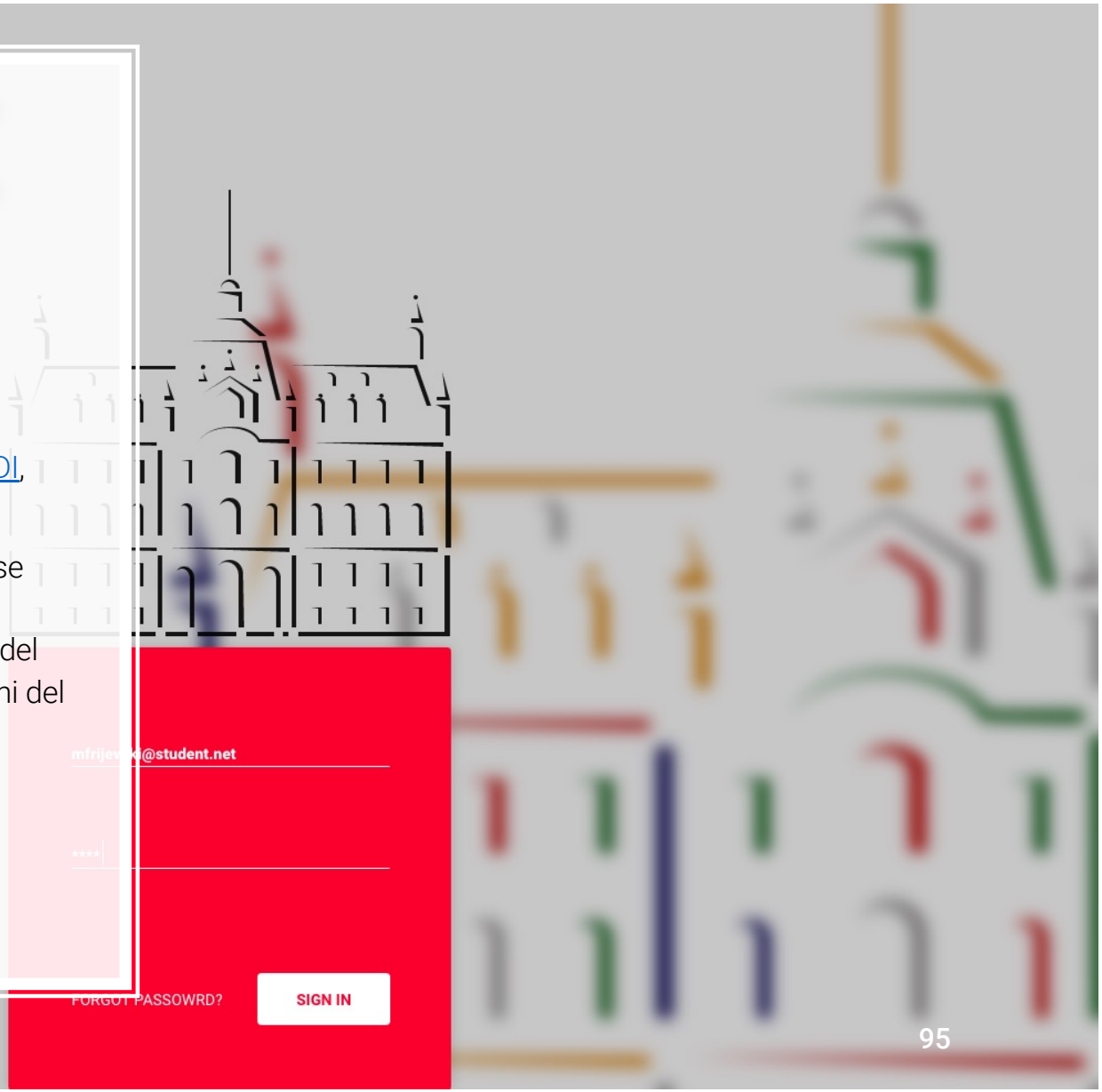
$$W^* = \underset{W}{\operatorname{argmax}}\, P(W|X) = \underset{W}{\operatorname{argmax}}$$

$P(X|W)$ acustic model lexicon

$P(W)$ language model

Projekt ON/Tolmač

- Razvoj novih modelov prevajalnika, predvidoma v ogrodju FB Fairseq ali NMT Marian
- Razvoj novih modelov razpoznave, predvidoma KALDI, NVIDIA Jasper, NVIDIA Nemo, Wave2letter
- Razvoj komponent arhitekture sistema. Uporabljajo se naslednje tehnologije:
  - Java Spring, Rabbit MQ, gRPC, Rest za zaledni del
  - React, React Native za spletne portale in mobilni del ter
  - Electorn za namizno aplikacijo,
  - Kubernetes za arhitekturo in orkestracijo.



Relevance feedback

- Improving IR based on user's feedback
- Rocchio method:
  - let $\boldsymbol{q}$ be original query vector and
  - D_r the set of relevant and D_{ir} the set of irrelevant documents.
  - The expanded query $\boldsymbol{q}_e$ can be calculated as:

$$\boldsymbol{q}_e = \alpha \boldsymbol{q} + \frac{\beta}{|D_r|} \sum_{d_r \in D_r} \boldsymbol{d}_r - \frac{\gamma}{|D_{ir}|} \sum_{d_{ir} \in D_{ir}} \boldsymbol{d}_{ir}$$

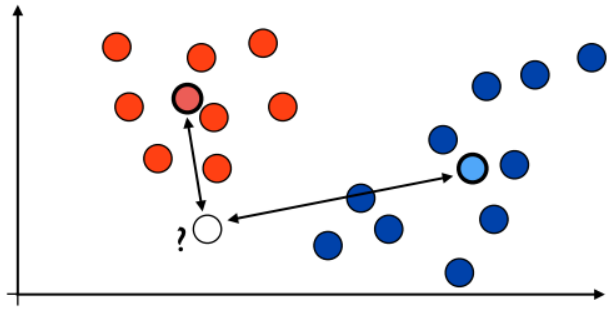
  - where α, β and γ parameters

Rocchio classification

- Supervised machine learning on feedback – no comparission with query.
- Prototype vectors c_i built for each class:

$$\boldsymbol{c_i} = \frac{\alpha}{|D_i|} \sum_{\boldsymbol{d} \in D_i} \frac{\boldsymbol{d}}{||\boldsymbol{d}||} - \frac{\beta}{|D - D_i|} \sum_{\boldsymbol{d} \in D - D_i} \frac{\boldsymbol{d}}{||\boldsymbol{d}||}$$

- where:
  - D_i set of documents of class i
  - α and β parameters



Other ML approaches

- LU learning
- PU learning
- Ranking SVM*

- Pseudo-relevance feedback

*Joachims, T. Optimizing search engines using clickthrough data. In Proceedings of ACM SIGKDD International Conference on Knowledge, Discovery and Data Mining (KDD-2002), 2002.

Word embeddings

- Representation of words in high-dimensional space
  - e.g. 300 – 500 dimensions
  - words with similar semantics/ usage appear closer...

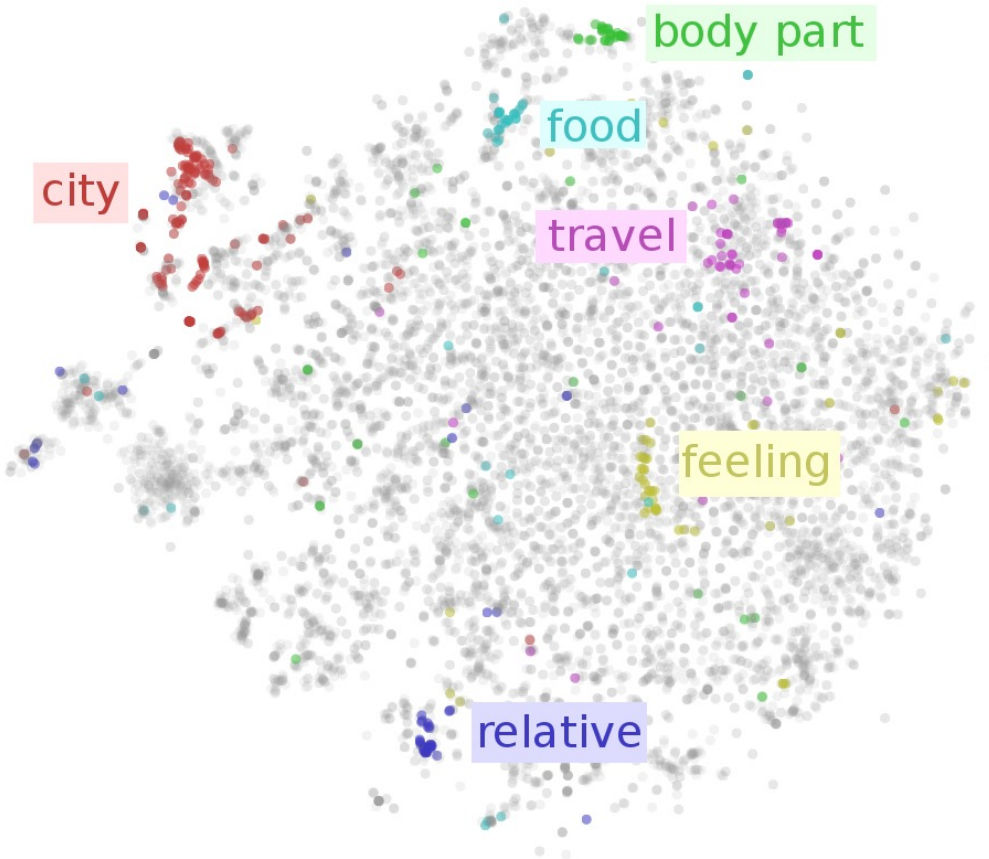


Image source: http://ruder.io/word-embeddings-1/

One hot encoding

- Encoding words with vocabulary words
  - vocabulary = {house, real-estate, rent, agency, provision, flat}
- One hot encoding

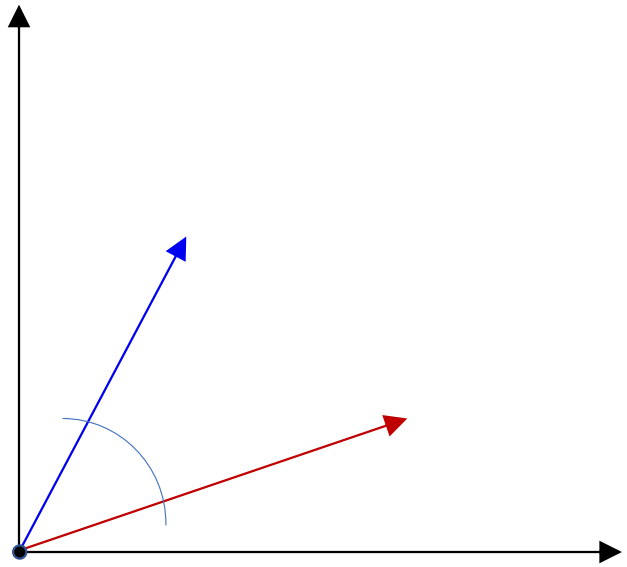
Word	v1	v2	v3	v4	v5	v6
house	1	0	0	0	0	0
real-estate	0	1	0	0	0	0
rent	0	0	1	0	0	0
agency	0	0	0	1	0	0
provision	0	0	0	0	1	0
flat	0	0	0	0	0	1

Document	v1	v2	v3	v4	v5	v6
real-estate agency	0	1	0	1	0	0

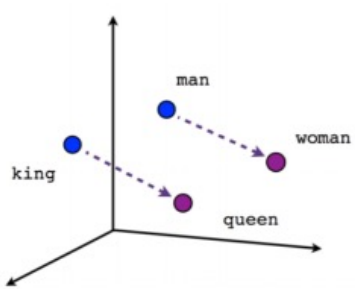
One hot encoding in vector space

- Corpus:
  - Romeo and Juliet.
  - Juliet: O happy dagger!
  - ...
- Vocabulary:
  - V = {romeo, juliet, happy, dagger,... },

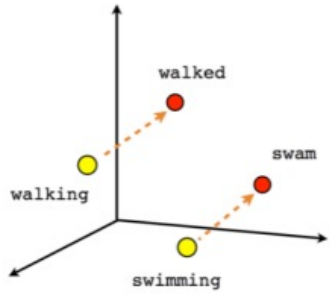
Category	w_1	w_2	...	$w_{\|V\|}$
romeo	1	0	...	0
juliet	0	1	...	0
...				



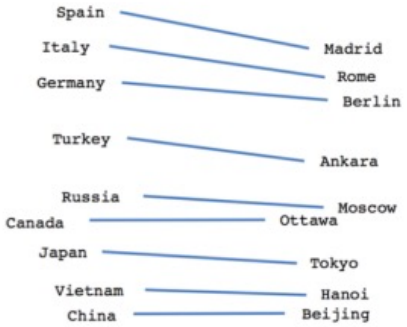
Word embeddings in vector space



Male-Female



Verb tense



Country-Capital

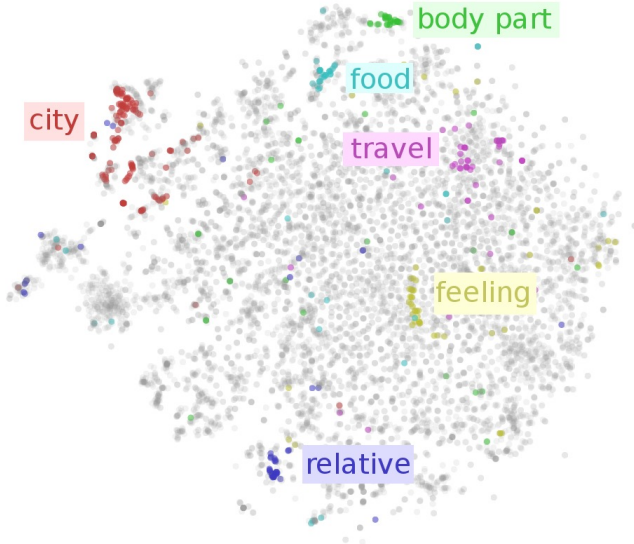
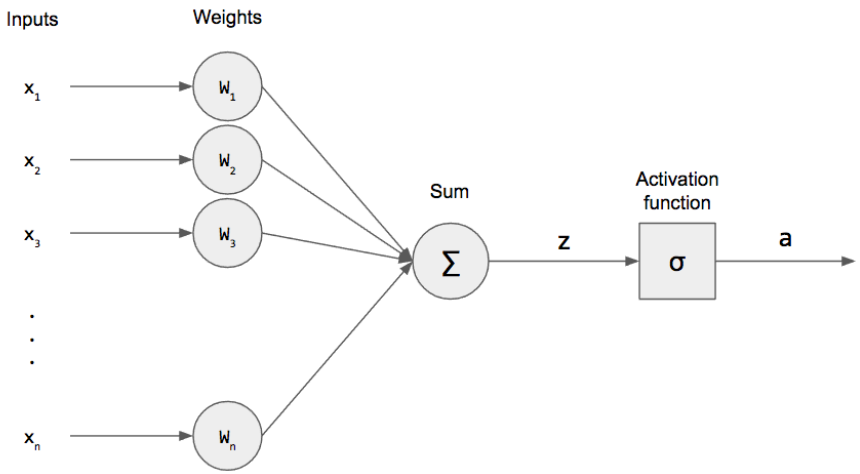


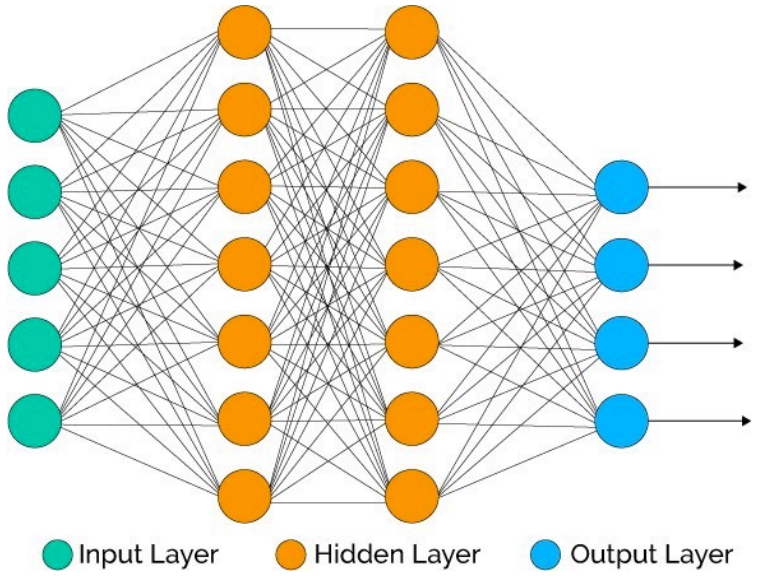
Image source: https://cbail.github.io/textasdata/word2vec/rmarkdown/word2vec.html

Neural networks

- Artificial neuron

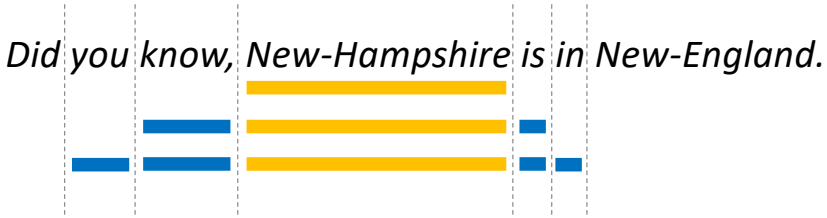


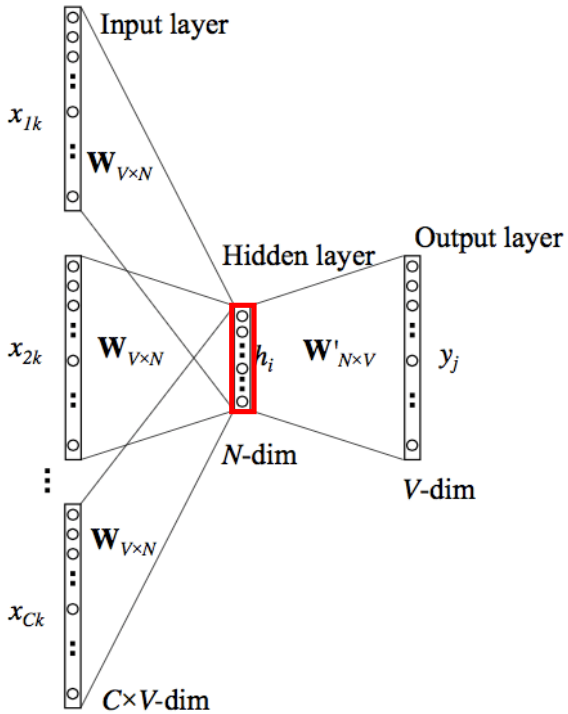
- Neural network



Learning word embeddings

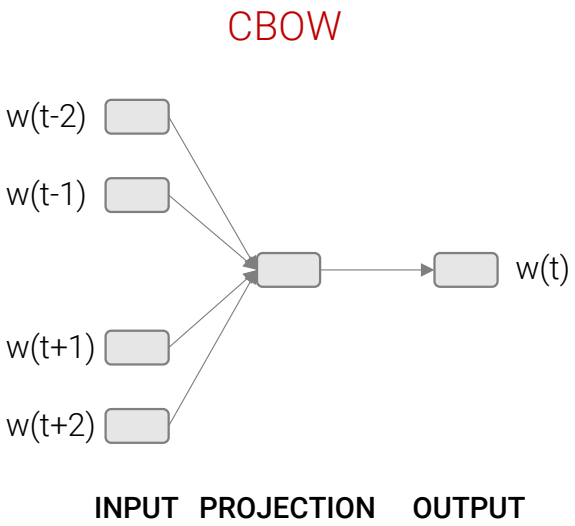
- Co-occurring words
- Context window

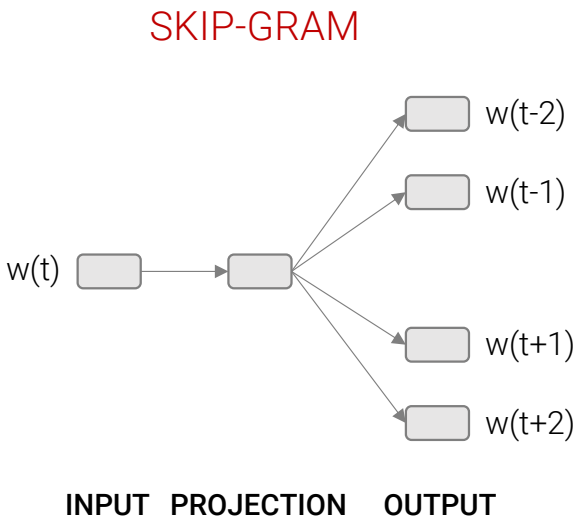




Embeddings models

- CBOW
- Skip-Gram





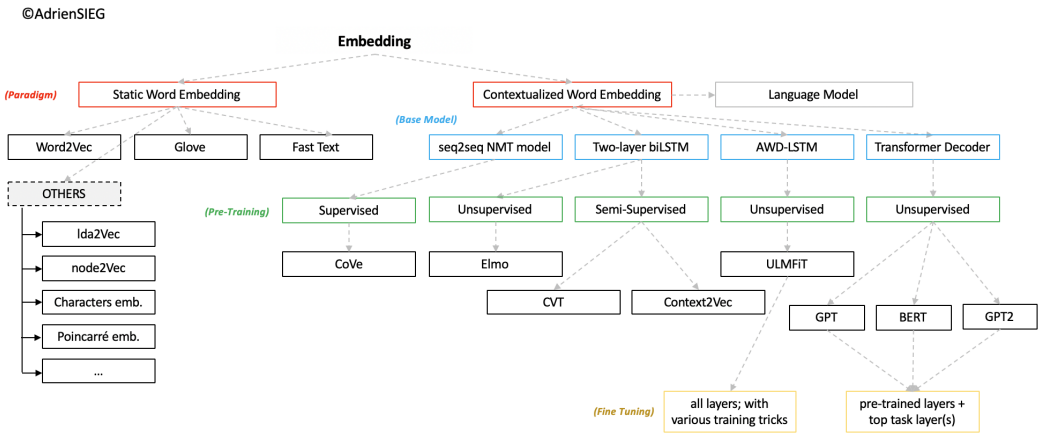
Word2Vec*

- Converts words to vectors using CBOW or SKIP-GRAM model implemented with shallow NN.
- Input:
  - text corpus
- Output:
  - word vector file

*T. Mikolov, K. Chen, G. Corrado, and J. Dean. *Efficient Estimation of Word Representations in Vector Space*. ArXiv e-prints, January 2013.

Other approaches

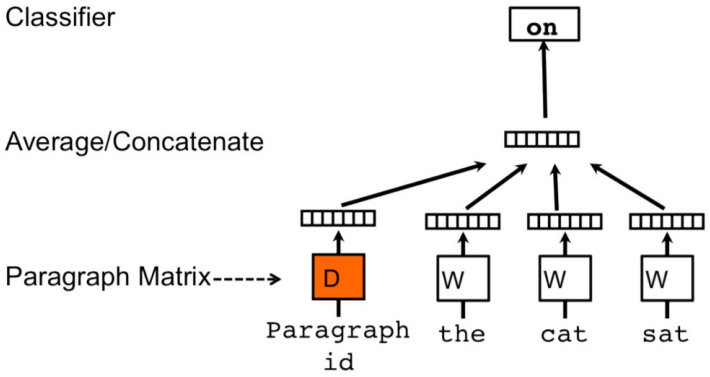
- Glove (Stanford, 2013)[1]
- FastText (Facebook, 2016)[2]
- ELMO
- BERT

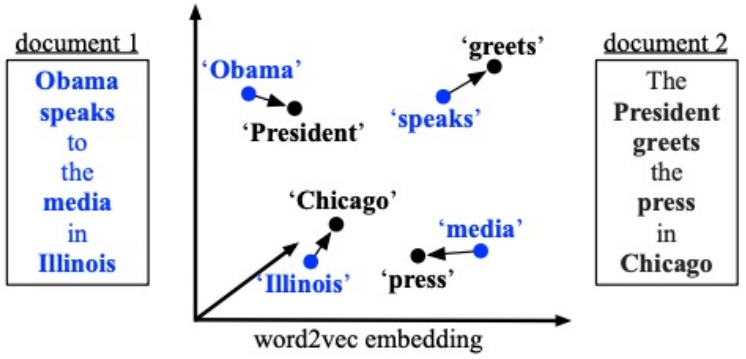


[1] https://nlp.stanford.edu/projects/glove/
[2] https://fasttext.cc/

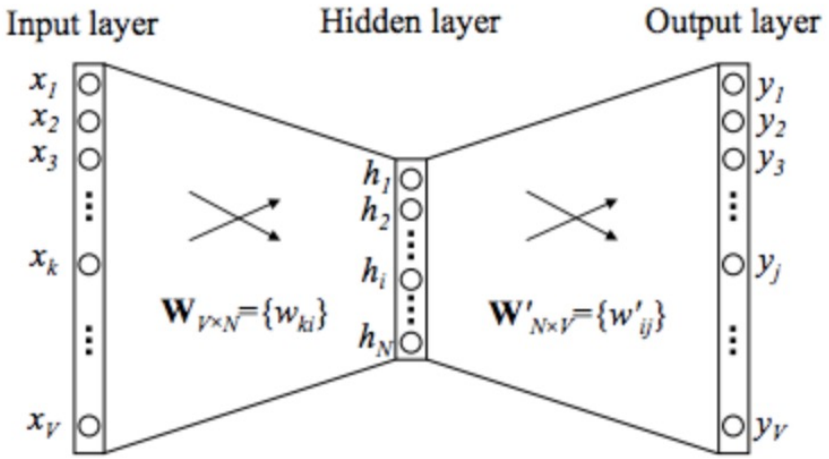
Using word embeddings in IR

- Word embeddings useful to retrieve similar words
- How to retrieve documents similar to a specific query?
- Approaches:
  - Weighted average of word embeddings
  - Word Mover's Distance (WMD)
  - Doc2Vec





Dual Embedding Space Model



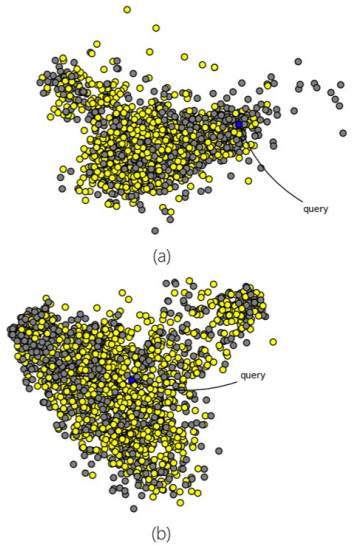


Figure 1: A two dimensional PCA projection of the 200-dimensional embeddings. Relevant documents are yellow, irrelevant documents are grey, and the query is blue. To visualize the results of multiple queries at once, before dimensionality reduction we centre query vectors at the origin and represent documents as the difference between the document vector and its query vector. (a) uses IN word vector centroids to represent both the query and the documents. (b) uses IN for the queries and OUT for the documents, and seems to have a higher density of relevant documents near the query.

*B. Mitra, E. Nalisnick, N. Craswell and R. Caruana. *A Dual Embedding Space Model for Document Ranking*. arXiv:1602.01137v1 [cs.IR] 2 Feb 2016.

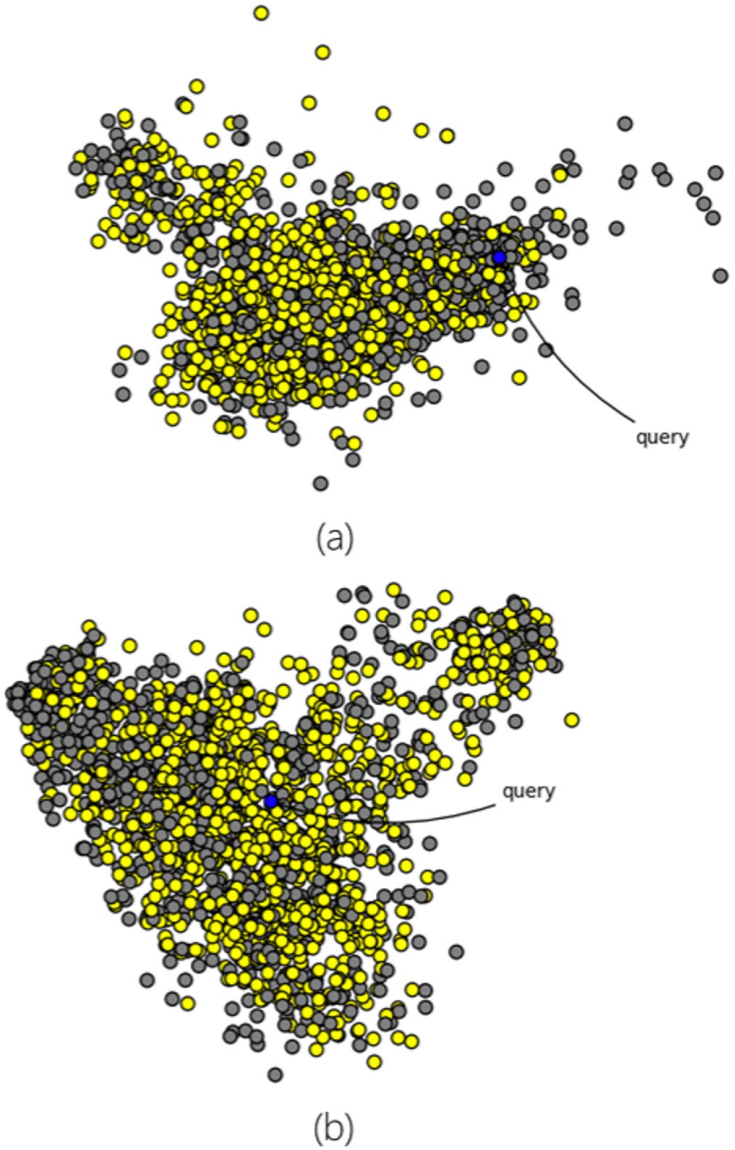
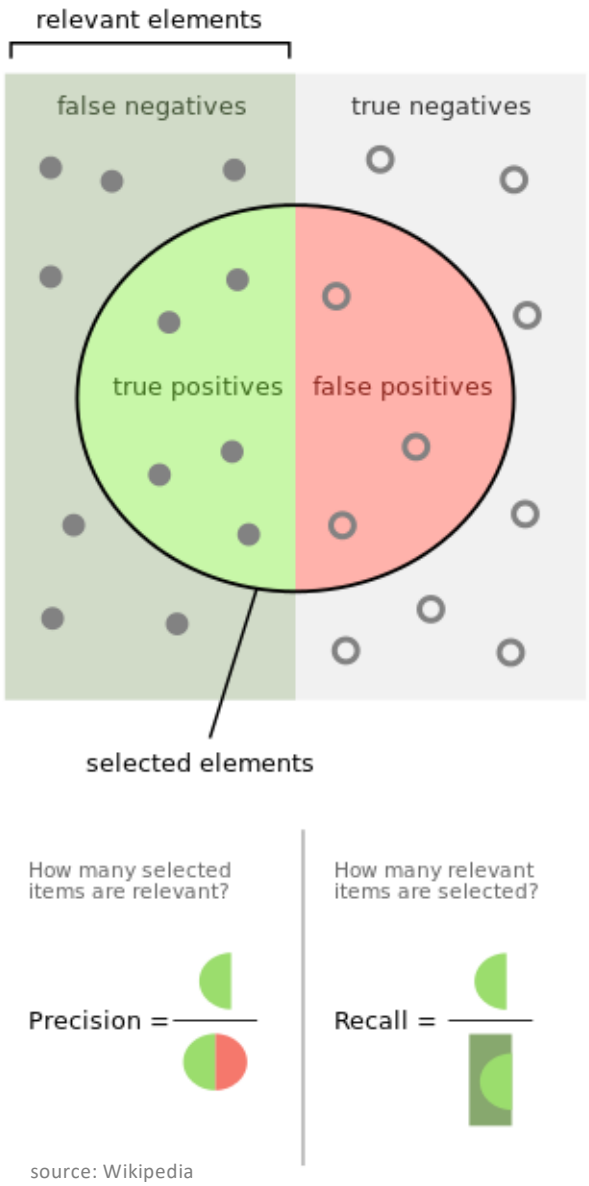


Table 1: The nearest neighbours for the words "yale", "seahawks" and "eminem" according to the cosine similarity based on the IN-IN, OUT-OUT and IN-OUT vector comparisons for the different words in the vocabulary. These examples show that IN-IN and OUT-OUT cosine similarities are high for words that are similar by function or type (*typical*), and the IN-OUT cosine similarities are high between words that often co-occur in the same query or document (*topical*). The *word2vec* model used here was trained on a query corpus with a vocabulary of 2,748,230 words.

yale			**seahawks**			**eminem**		
IN-IN	OUT-OUT	IN-OUT	IN-IN	OUT-OUT	IN-OUT	IN-IN	OUT-OUT	IN-OUT
yale	yale	yale	seahawks	seahawks	seahawks	eminem	eminem	eminem
harvard	uconn	faculty	49ers	broncos	highlights	rihanna	rihanna	rap
nyu	harvard	alumni	broncos	49ers	jerseys	ludacris	dre	featuring
cornell	tulane	orientation	packers	nfl	tshirts	kanye	kanye	tracklist
tulane	nyu	haven	nfl	packers	seattle	beyonce	beyonce	diss
tufts	tufts	graduate	steelers	steelers	hats	2pac	tupac	performs

Evaluation measures

- In ML, we use precision and recall
- In IR and web search we rank not classify!



Evaluation of ranking

- Let D be a collection of documents with $|D| = N$.
- For a query q the retrieval algorithm produces ranking $R_q: \langle d_1^q, d_2^q, \ldots, d_N^q \rangle$, where $d_1^q \in D$ most relevant and $d_N^q \in D$ most irrelevant document to q.
- P and R can be calculated for each rank position:
  - Recall at rank position $i: \; r(i) = \frac{S_i}{|D_q|}$,
  - Precision at rank position $i: \; p(i) = \frac{S_i}{i}$

$D_q \subseteq D$: set of actual relevant documents
S_i: number of relevant documents from position 1 to i.

Example of ranking at each position

Rank i	+/–	p(i)	r(i)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	-	7/11 = 64%	7/8 = 88%
12	+	8/12 = 67%	8/8 = 100%

Average precision

Rank i	+/–	p(i)	r(i)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	-	7/11 = 64%	7/8 = 88%
12	+	8/12 = 67%	8/8 = 100%

- Average precision:

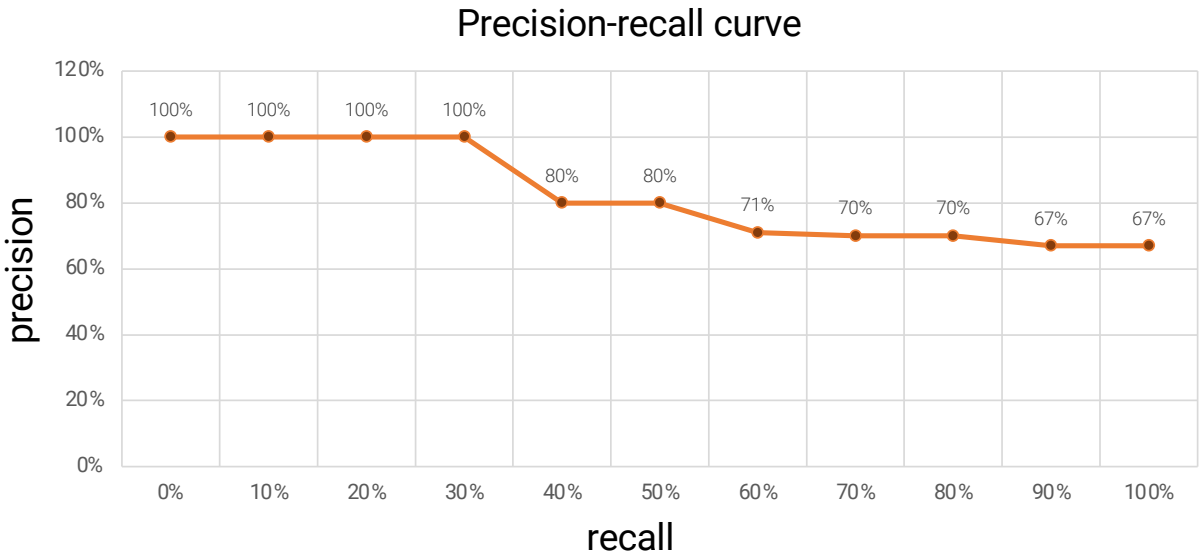
$$p_{avg} = \frac{\sum_{d_i^q \in D_q} p(i)}{|D_q|}$$

- $p_{avg} = \frac{100\% + 100\% + 100\% + 80\% + 71\% + 67\% + 70\% + 67\%}{8} = 82\%$

Precision-recall curve

- Plotted at standard recall levels: 0%, 10%, 20%, ..., 100%
- $p(r_i) = \max_{r_i \leq r \leq r_{10}} p(r)$

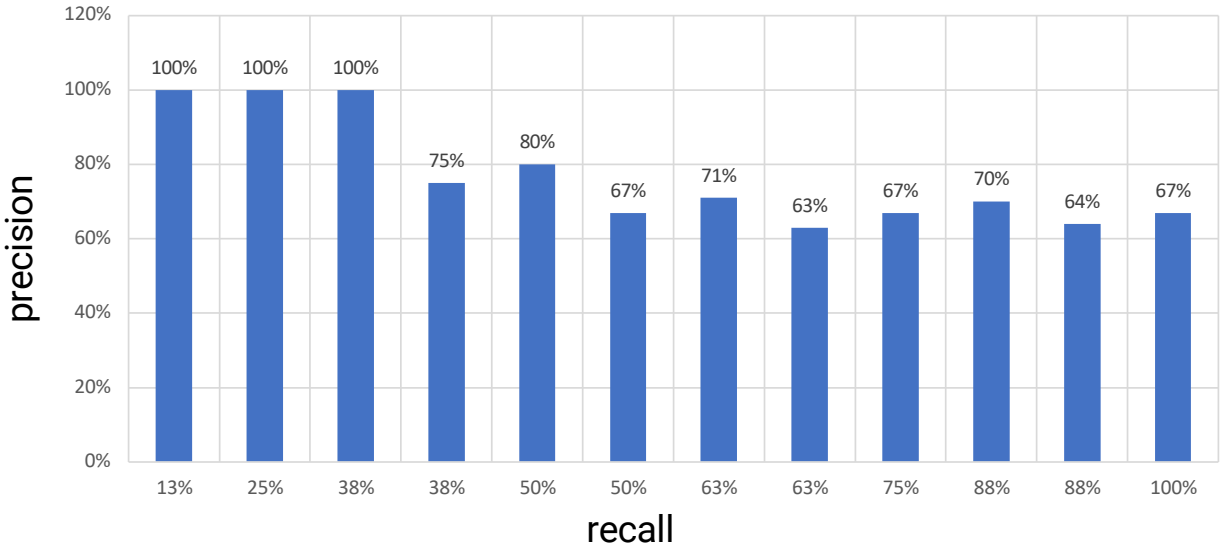
i	Relevant	$p(r_i)$	r_i
1	+	100%	13%
2	+	100%	25%
3	+	100%	38%
4	-	75%	38%
5	+	80%	50%
6	-	67%	50%
7	+	71%	63%
8	-	63%	63%
9	+	67%	75%
10	+	70%	88%
11	-	64%	88%
12	+	67%	100%



Precision-recall curve

- Precision at rank positions

i	Relevant	$p(r_i)$	r_i
1	+	100%	13%
2	+	100%	25%
3	+	100%	38%
4	-	75%	38%
5	+	80%	50%
6	-	67%	50%
7	+	71%	63%
8	-	63%	63%
9	+	67%	75%
10	+	70%	88%
11	-	64%	88%
12	+	67%	100%



Evaluation on multiple queries

- Overall precision on multiple queries:

$$\bar{p}(r_i) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} p_j(r_i)$$

- where:
  - $|Q|$ number of queries,
  - $p_j(r_i)$ precision q_j at the recall level r_i

Inverted index

- Most efficient index for IR
- Consists of:
  - vocabulary V of distinctive terms of the document set $D = \{d_1, d_2, \dots, d_n\}$
  - for each term t_i an inverted index of postings.

$$t_i \rightarrow \langle posting_1 \rangle, \langle posting_2 \rangle, \dots, \langle posting_n \rangle$$

- Posting: doc id + other relevant information for retrieval

Example

- Example of inverted index for proximity search:
- $< id_j, f_{ij}, [o_1, o_2, \ldots, o_{|f_{ij}|}] >$ where
  - f_{ij} - frequency of t_i in d_j
  - o_i - offset of t_i in d_j

Corpus:

id_1: Web mining is useful.
1 2 3 4

id_2: Usage mining applications.
1 2 3

id_3: Web structure mining studies the Web hyperlink structure.
1 2 3 4 5 6 7 8

Vocabulary:

{web, mining, useful, applications, usage, structure, studies, hyperlink}

Index:

- applications: $\langle id_2, 1, [3] \rangle$
- hyperlink: $\langle id_3, 1, [7] \rangle$
- mining: $\langle id_1, 1, [2] \rangle, \langle id_2, 1, [2] \rangle, \langle id_3, 1, [3] \rangle$
- structure: $\langle id_3, 2, [2, 8] \rangle$
- studies: $\langle id_3, 1, [4] \rangle$
- usage: $\langle id_2, 1, [1] \rangle$
- useful: $\langle id_1, 1, [4] \rangle$
- web: $\langle id_1, 1, [1] \rangle, \langle id_3, 2, [1, 6] \rangle$

Search on inverted index

- Algorithm:

```
for each query term q_i ∈ Q
   find q_i in vocabulary V
   retrieve list list_i of docs d_j where d_j ∈ D and q_i ∈ dj
end
merge doc lists list_i
for each doc d_j ∈ merge_list
   compute rank score
end
print ranked list of docs
```

Example

- Which document gets the highest rank in the proximity search for Q = web mining?

Corpus:

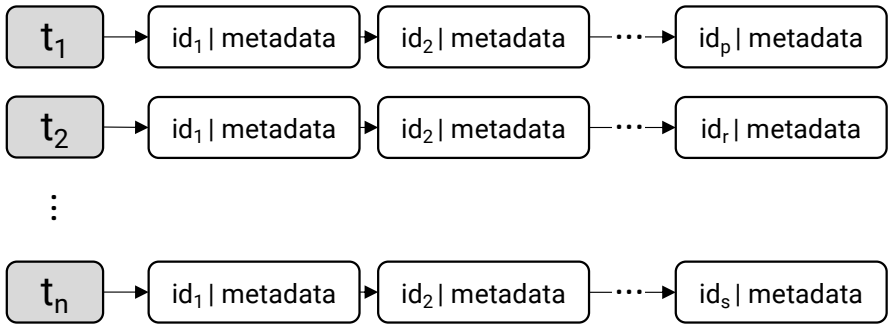
id_1: Web mining is useful.
id_2: Usage mining applications.
id_3 : Web structure mining studies the Web hyperlink structure.

Index:

- applications: $\langle id_2, 1, [3] \rangle$
- hyperlink: $\langle id_3, 1, [7] \rangle$
- mining: $\langle id_1, 1, [2] \rangle, \langle id_2, 1, [2] \rangle, \langle id_3, 1, [3] \rangle$
- structure: $\langle id_3, 2, [2, 8] \rangle$
- studies: $\langle id_3, 1, [4] \rangle$
- usage: $\langle id_2, 1, [1] \rangle$
- useful: $\langle id_1, 1, [4] \rangle$
- web: $\langle id_1, 1, [1] \rangle, \langle id_3, 2, [1, 6] \rangle$

Query: web mining

Index construction

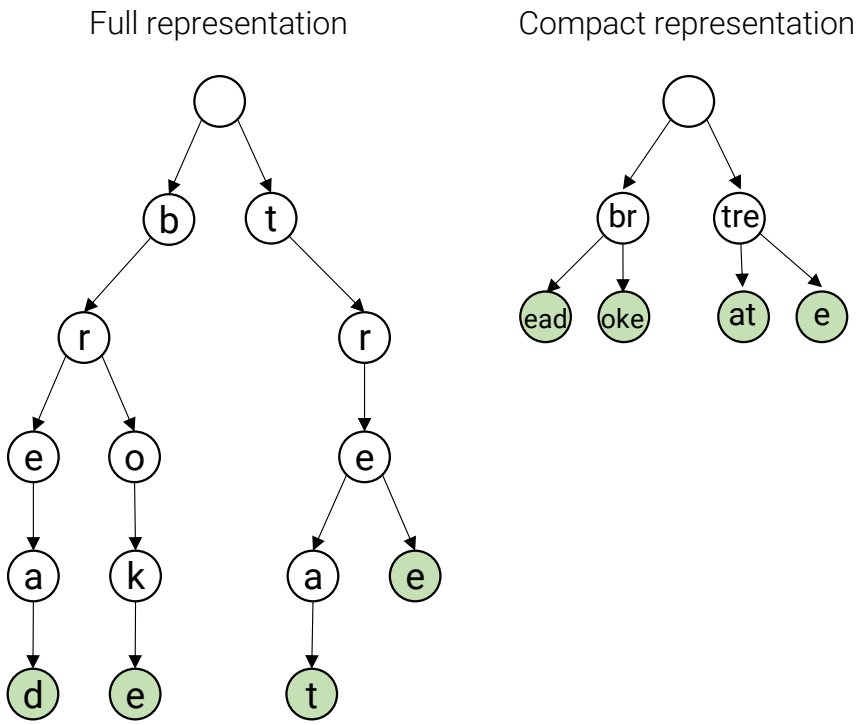


- Algorithm:

```
for each doc d_j ∈ D
   for each term t ∈ d_j
         find position e of t in index
         if t not found then create new element e in index
         add (ID, metadata) of d_j to index at position e
   end
end
```

Trie data structure

- Trie – retrieval
- Prefix tree with ordered content
- Nodes: string characters
- Example:
  - words: bread, broke, treat, tree
- Visualization:
  https://www.cs.usfca.edu/~galles/visualization/Trie.html



Example

- Build trie data structure for corpus: id_1, id_2, id_3
- Vocabulary:

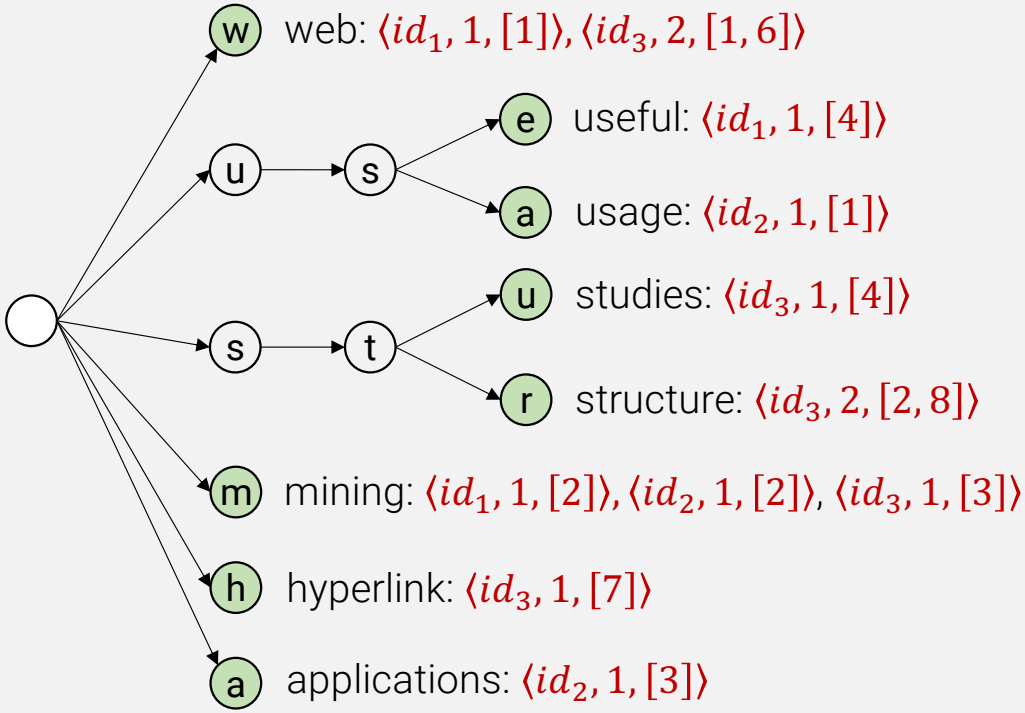
  {applications, hyperlink, mining, structure, studies, usage, useful, web}

Corpus:

id_1: Web mining is useful.
1 2 3 4

id_2: Usage mining applications.
1 2 3

id_3: Web structure mining studies the Web hyperlink structure.
1 2 3 4 5 6 7 8



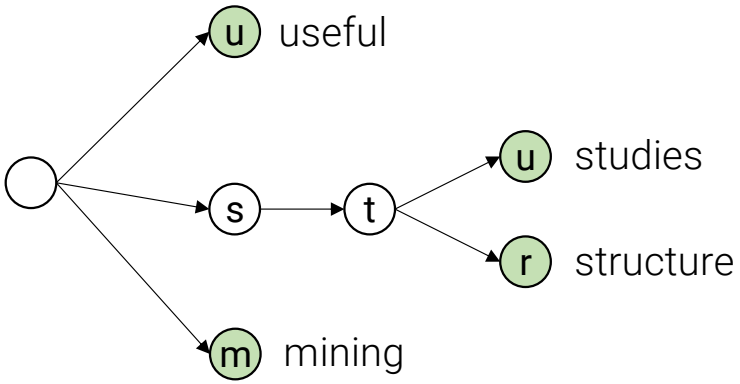
Time complexity

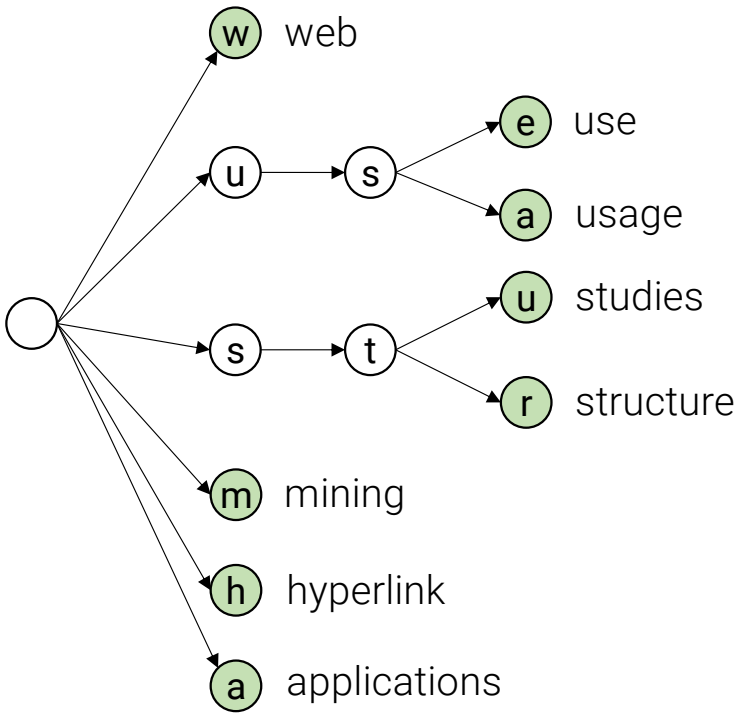
- Trie construction
  - Linear: $O(T)$, where T number of terms including duplicates
- Trie search
  - Linear: $O(n)$, where n length of a retrieved term

Trie construction for Web

- In Web environment, trie becomes very large.
- Changes in the construction algorithm:
  - write to a partial index I_i
  - when memory full, write partial index to disk
  - continue with new partial index
  - when all documents processed, merge indices:
    - pairwise (i.e. I_{1-2}, I_{3-4}, ..., I_{n-n+1} then I_{12-34}, ...) or
    - step by step (i.e. I_{1-2}, I_{12-3}, I_{123-4}, ...)

Trie merging





Additional indices

- Web dynamic, pages constantly added, deleted
- Additional indices:
  - index of added pages
  - index of deleted pages
- Algorithm:
  - search in main index → D_0
  - search in index of added pages → D_+
  - search in index of deleted pages → D_-
  - result: $(D_0 \cup D_+) - D_-$

Index compression

- Index compression needed to fit entire index or its large part to memory...
- Content of index represented with integers
- Basic idea:
  - Integers represented with 4 bytes (32 bits)
  - Many integers smaller than 4 bytes – use integer coding.
  - Doc ids can be very large numbers – use gaps (id offsets) instead.
    - Example, IDs: 4, 10, 300, and 305. Represented with offset, 4, 6, 290 and 5. Frequent terms have smaller gaps!

Types of integer coding

Coding type	Coding	Rules
Variable-bit scheme	Unary coding	for int x, prefix 1 with $x-1$ zeroes
	Elias Gamma coding	for int x, prefix 1 with $\lfloor log_2 x \rfloor$ zeroes and add x in binary without the most significant bit
	Elias Delta coding	for int x, represent $1+\lfloor log_2 x \rfloor$ in Gamma code and add x in binary without the most significant bit.
	Golomb coding	for int x, represent $\mathrm{q} = \left\lfloor \frac{x}{b} \right\rfloor + 1$ in unary and add binary representation of $r = x - qb$ using coding tree.
Variable-byte scheme	Variable-byte coding	Use seven bits in each byte to code int x, with the least significant bit set to 1 in the last byte, or to 0 if further bytes follow.

Exercise

- Represent number 5 in variable-bit coding.

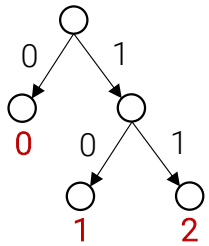
Coding	Rules
Unary coding	for int x, prefix 1 with $x-1$ zeroes
Elias Gamma coding	for int x, prefix 1 with $\lfloor log_2 x \rfloor$ zeroes and add x in binary without the most significant bit
Elias Delta coding	for int x, represent $1+\lfloor log_2 x \rfloor$ in Gamma code and add x in binary without the most significant bit.
Golomb coding	for int x, represent $q = \lfloor \frac{x}{b} \rfloor + 1$ in unary and add binary representation of $r = x - qb$ using coding tree.
Variable-byte coding	Use seven bits in each byte to code int x, with the least significant bit set to 1 in the last byte, or to 0 if further bytes follow.

Unary: 00001

Elias gamma: $\text{concat}(001, 01) = 00101$
$\lfloor log_2 5 \rfloor = 2, \text{binary}(5) = 101$

Elias Delta: $\text{concat}(011, 01) = 01101$
$1 + \lfloor log_2 5 \rfloor = 3, \text{gamma}(3) = 011, \text{binary}(5) = 101$

Golomb (b=3): $\text{concat}(01, 11) = 0111$
$\text{unary}(\lfloor \frac{5}{3} \rfloor + 1) = 01, \text{r} = 5 - \lfloor \frac{5}{3} \rfloor 3 = 2, \text{i} = \lfloor log_2 3 \rfloor = 1,$
$d = 2^{i+1} - b = 4 - 3 = 1, coding\_tree(r) = coding_{tree}(2) = 11$



Exercise

- Represent number 135 in variable-byte coding.

135 with 4-bytes binary: 00000000 00000000 00000000 10000111

135 with variable-byte coding: 00000011 00001110

Decoding...

- Elias Gamma
  - count zeroes - K
  - consider first one after K zeroes as the first digit of the integer, i.e. 2^K
  - read the remaining K bits of the integer.
- Elias Delta
  - count zeroes - L
  - consider first one after L zeroes as the first digit of the integer, i.e. 2^L
  - read the remaining L bits of the integer M.
  - use $M - 1$ bits and add one on the first place.

Decoding

- Golomb
  - decode unary-coded quotient q.
  - compute $i = \lfloor log_2 b \rfloor$ and $d = 2^{i+1} - b$.
  - retrieve the next i bits and assign it to r.
  - if $r \geq d$ then
    - retrieve one more bit and append it to r at the end;
    - $r = r - d$.
  - return $x = qb + r$.
- Variable byte
  - read all bytes until a byte with the zero last bit is seen.
  - remove the least significant bit from each byte read so far and concatenate the remaining bits.

Exercise

- count zeroes - L
- consider first one after L zeroes as the first digit of the integer, i.e. 2^L
- read the remaining L bits of the integer M.
- use $M - 1$ bits and add one on the first place.

- Decode 00100001 written in Elias Delta coding.

Number of zeroes: 2

M = first one and remaining 2 bits: $100 = 4$

Result = one plus $M - 1$ leftmost bits: $1001 = 9$

Empirical tests

- Unary – nonefficient for large numbers, space consuming
- Parametrized Golomb more space efficient and faster for IR than non-parameterized Elias coding.
- Variable-byte integers often faster than variable-bit integers (fewer CPU operations required for decoding).
- A suitable compression allow retrieval to be up to twice as fast than without compression, while the space requirement averages 20% – 25% of the cost of storing uncompressed integers.

Latent Semantic Indexing

- IR models typically based on keyword matching.
- Same concepts can be described with different words (synonyms)... important docs might not be retrieved.
- Latent Semantic Indexing* (LSI) identifies statistical associations among terms.
- Based on Singular Value Decomposition (SVD).

*Deerwester, S., S. Dumais, G. Furnas, T. Landauer, and R. Harshman. *Indexing by latent semantic analysis*. Journal of the American Society for Information Science, 1990, 41(6): p. 391-407.

Formalization

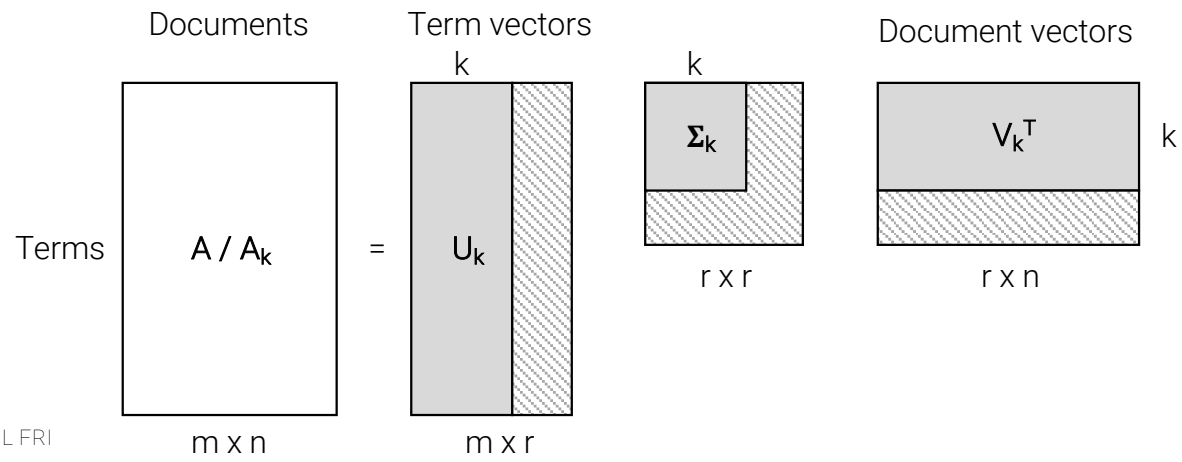
- Let D be the document collection, with m distinctive words and size $|D| = n$.
- Size of term-document matrix A is $m \times n$.
- Let A_{ij} be computed as TF, i.e. the number of times term i occurs in document j.

Singular Value Decomposition

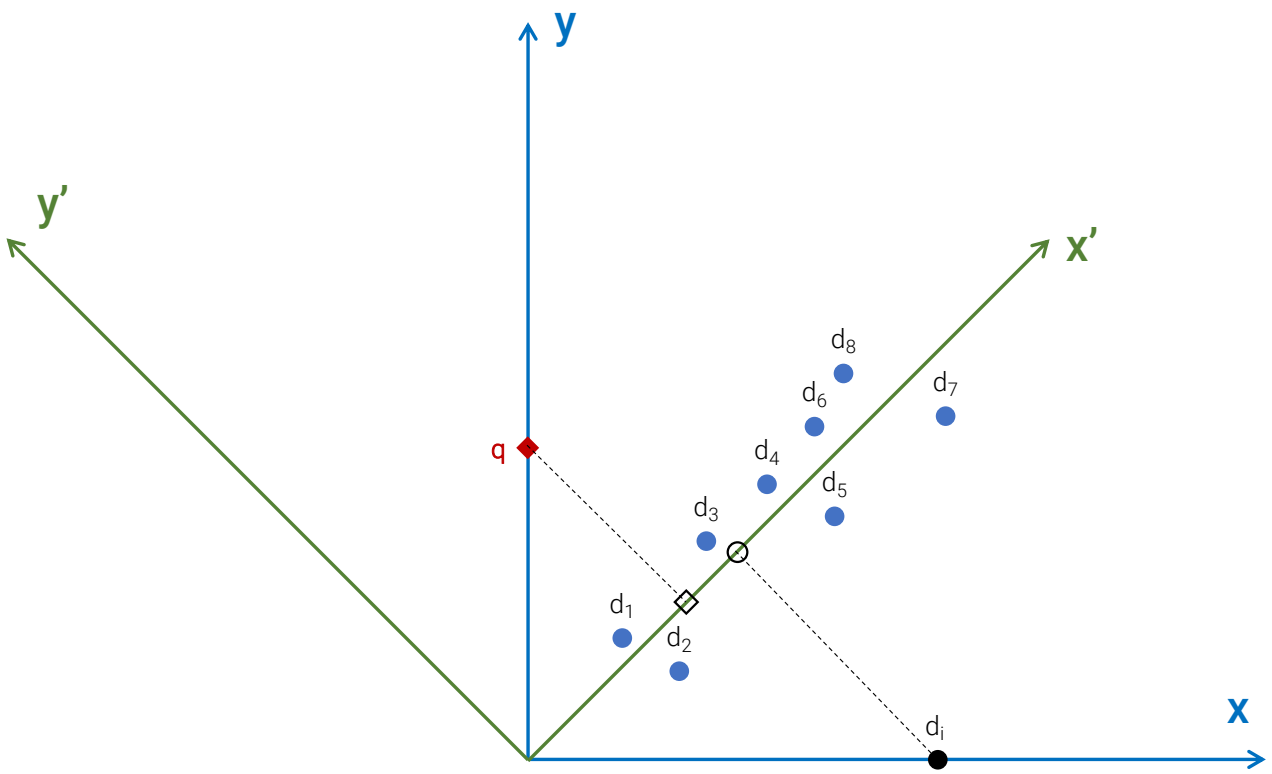
- SVD factors A into the product of 3 matrices: $A = U\Sigma V^T$, where:
  - U is a $m \times r$ matrix with columns that are unit orthogonal vectors called left singular vectors. They are eigenvectors associated with the r non-zero eigenvalues of AA^T.
  - V is a $n \times r$ matrix with columns that are unit orthogonal vectors called right singular vectors. They are eigenvectors associated with the r non-zero eigenvalues of $A^T A$.
  - Σ is a $r \times r$ diagonal matrix, $\Sigma = diag(\sigma_1, \sigma_2, \dots, \sigma_r)$, $\sigma_i > 0$. σ_i called singular values, are the non-negative square roots of the r eigenvalues of AA^T (arranged in decreasing order).
  - r represents rank of A, $r \leq \min(m, n)$

Reducing hidden concept space

- Matrix Σ can be reduced by removing dimensions associated with smallest singular values.
- Let's say we use only k largest singular values in Σ. This leads us to the k-concept space: $A_{k=}U_k\Sigma_k V_k^T$



Intuition of LSI



Query and retrieval using LSI

- q treated as a document and transformed to k-concept space
- Transformed documents are represented with V_k.
- q_k becomes an additional column/document in V_k^T. Thus q is:

$$q = U_k \Sigma_k q_k^T$$

- and q_k is:

$$q_k = q^T U_k \Sigma_k^{-1}$$

- For retrieval, q_k is compared with other documents in V_k.

Exercise...

- For the below corpus use LSI to find best matching documents for the query q = *dies, dagger*.
- Corpus:
  - d_1: *Romeo and Juliet.*,
  - d_2: *Juliet: O happy dagger!*,
  - d_3: *Romeo died by dagger.*,
  - d_4: *'Live free or die', that's the New-Hampshire's motto.*,
  - d_5: *Did you know, New-Hampshire is in New-England*.

Exercise

- Procedure:
  - pre-process corpus (punctuation/stopwords removal, lemmatization, lowercase...)
  - define vocabulary $vocab$
  - create TF matrix: A for vocabulary $vocab$
  - decompose A using SVD: $A = U\Sigma V^T$
  - reduce dimensions of the SVD matrices to the size k: $A_k = U_k \Sigma_k V_k^T$
  - calculate query representation in k-concept space: $q_k = q^T U_k \Sigma_k^{-1}$
  - for each document d_i as represented in V^T, calculate cosine similarity with q_k.

LSI vs keywords-based methods

- LSI performs better than traditional keywords-based methods.
- Drawbacks:
  - Time complexity of SVD ($O(m^2 n)$) too high for large corpuses.
  - Concept space not interpretable.
  - Finding optimal k for dimension reduction hard problem.
- LSI vs word embeddings

Web search

- Web search mainly based on the vector space model and term matching.
- Main steps behind web searching:
  - crawling
  - parsing
  - indexing
  - searching and ranking

Ranking

- Ranking most important feature of search engines.
- Traditional approaches such as cosine similarity do not suffice.
- For Web, quality of pages also important factor:
  - quality of pages on the Web differs a lot…
  - hyperlinks can be exploited to measure how important is a specific page.
- Both, page content and reputation usually considered when ranking pages.

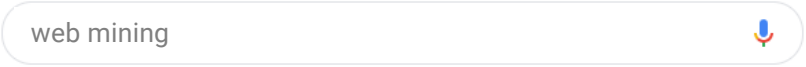
Content based ranking

- Query terms can occur in several places in a page. Different kind of info is considered:
  - Occurrence type: title, anchor text, URL, body
  - Count: for each type of occurrence
  - Position: for each type of occurrence – used for proximity evaluation in case of multiple query terms
- Weights are assigned to occurrence types and counts:
  - occurence type vector: $\overrightarrow{otv} = [tw_{title}, tw_{anchor}, tw_{url}, tw_{body}]$
  - count vector: $\overrightarrow{cv} = [cw_{title}, cw_{anchor}, cw_{url}, cw_{body}]$

Single word queries

- Procedure:
  - define occurrence type vector $\overrightarrow{otv}$
  - for each page p retrieved from the inverted index
    - calculate count vector $\overrightarrow{cv}$
    - calculate IR score: $\overrightarrow{cv} \circ \overrightarrow{otv}$
    - calculate Reputation score
    - calculate Page score = $f(IR\ score, Reputation\ score)$

Multiple word queries



- Additionally to consider:
  - term proximity
  - term ordering
- Procedure (ignoring term ordering):
  - define type proximity vector $\overrightarrow{tpv}$
  - for each page p retrieved from the inverted index
    - for each pair of query terms calculate proximity value pv
    - calculate count vector $\overrightarrow{cv}$ for each occurrence type and proximity value
    - calculate IR score: $\overrightarrow{cv} \circ \overrightarrow{tpv}$
    - calculate Reputation score
    - calculate Page score = $f(IR\ score, Reputation\ score)$

Evaluation of page reputation

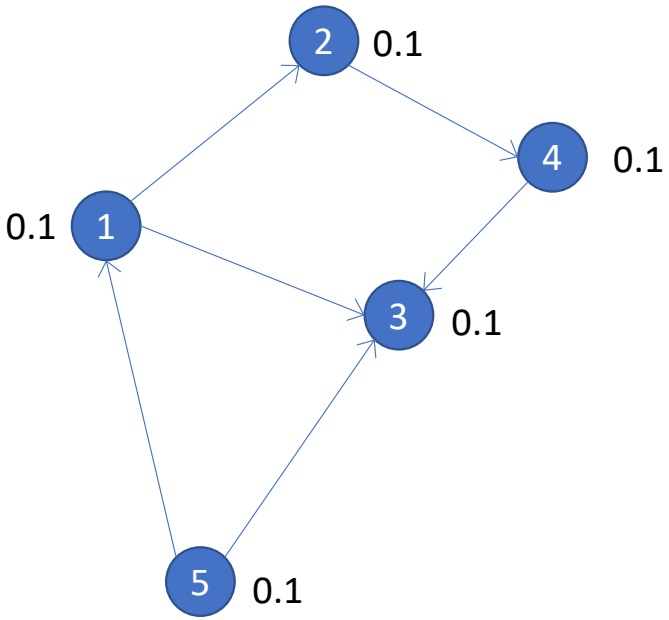
- Page reputation computed based on the link structure of the page.
- Two algorithms very important: PageRank and HITS.
- Both introduced in 1998:
  - HITS presented by Jon Kleinberg,
  - PageRank presented by Sergey Brin and Larry Page. Based on the algorithm, they built the search engine Google.

Prestige

- PageRank and HITS based on prestige.
- Prestige types:
  - Degree prestige: an actor is prestigious if it receives many in-links: $P_D(i) = \frac{d_I(i)}{n-1}$, where $d_I(i)$ is the in-degree of i and n is the total number of actors in the network.
  - Proximity prestige: only actors that are directly or indirectly connected to i are considered (influence domain): $P_P(i) = \frac{|I_i|/(n-1)}{\sum_{j \in I_i} d(j,i)/|I_i|}$, where $|I_i|/(n-1)$ is the proportion of actors that can reach actor i.

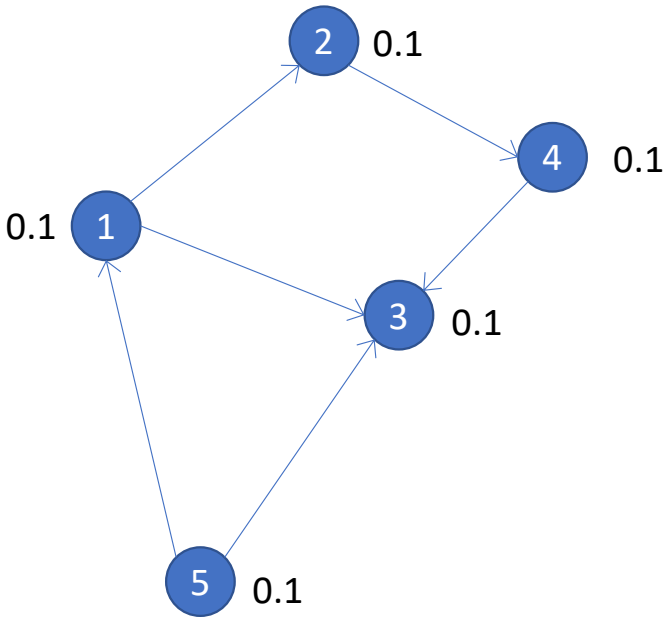
Rank prestige

- Rank prestige: considers the prominence of actors that "vote" for the actor under observation.
  - Rank prestige $PR(i)$ is defined as a linear combination of links that point to i: $P_R(i) = \sum_j P_r(j)$, where page j points to page i.
  - Let $\boldsymbol{P}$ represent the vector with rank prestige values of all actors: $\boldsymbol{P} = (P_R(1), P_R(2),\ldots, P_R(n))^T$ and A matrix where A_{ij}=1 if i points to j or 0 otherwise.
  - We then have $\boldsymbol{P} = \boldsymbol{A}^T\boldsymbol{P}$ where $\boldsymbol{P}$ is eigenvector of $\boldsymbol{A}^T$ and can be calculated using well known Power iteration method.
  - Note: Power iteration not directly useful for PageRank calculation, since A does not fulfil required preconditions.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$P = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$P_1 = A^T * P_0$$

$$P_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.1 \\ 0.0 \end{bmatrix}$$

PageRank

- The basic idea of PageRank follows the idea of rank prestige with the following difference:
  - Since a page may point to many other pages, its prestige score should be shared among all the pages that it points to. Thus
  - $P(i) = \sum_{(j,i)\in E} \frac{P(j)}{O_j}$, where Web is represented as a directed graph $G = (V, E)$ with V as a set of nodes and E as set of edges or hyperlinks and O_j is the number of out-links of j. Let n be the total number of pages, i.e. $n = |V|$.

$$A_{ij} = \begin{cases} \frac{1}{O_i} \; if \; (i,j) \in E \\ 0 \; otherwise \end{cases}, P = (P(1), P(2), \dots, P(n))^T, P = A^T P.$$

Modeling Web with Markov chain...

- Markov chain models Web surfing as a stochastic process where the surfer randomly and uniformly choses where to go next.
- We can calculate the probability of a random surfer to be in state j after one step as follows: $p_1(j) = \sum_{i=1}^{n} A_{ij}(1)\, p_0(i)$, where $A_{ij}(1)$ is the probability of going from i to j in 1 transition, and $A_{ij}(1) = A_{ij}$. In matrix form: $\boldsymbol{p_1} = \boldsymbol{A^T p_0}$.
- The probability distribution after k steps is $\boldsymbol{p_k} = \boldsymbol{A^T p_{k-1}}$.

Ergodic Theorem of Markov chains

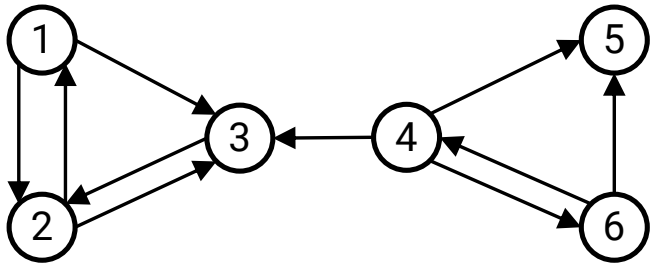
- By the Ergodic Theorem of Markov chains, a finite Markov chain defined by the stochastic transition matrix A has a unique stationary probability distribution if A is irreducible and aperiodic.
- Stationary probability distribution: after a series of transitions $\boldsymbol{p_k}$ will converge to a steady-state probability vector $\boldsymbol{\pi}$ regardless of the choice of the initial probability vector $\boldsymbol{p_0}$, i.e.: $\lim_{k\to\infty} \boldsymbol{p_k} = \boldsymbol{\pi}$
- When in steady state, $\boldsymbol{p_k} = \boldsymbol{p_{k+1}} = \boldsymbol{\pi}$ which means $\boldsymbol{\pi} = \boldsymbol{A^T\pi}$ and $\boldsymbol{\pi}$ is the principal eigenvector of $\boldsymbol{A^T}$ with eigenvalue of 1.

Stationary probability distribution

- Matrix A has to be:
  - Stochastic: a stochastic matrix is the transition matrix for a finite Markov chain whose entries in each row are non-negative real numbers and sum to 1.
  - Irreducible: irreducible means that the Web graph G is strongly connected.
  - Aperiodic: a state i is periodic with period $k > 1$ if k is the smallest number such that all paths leading from state i back to state i have a length that is a multiple of k. If a state is not periodic (i.e., k = 1), it is aperiodic. A Markov chain is aperiodic if all states are aperiodic.
- When modeling Web with Markov chain, neither of these requirements is satisfied.

Stochastic matrix

- Example of non stochastic Web subgraph

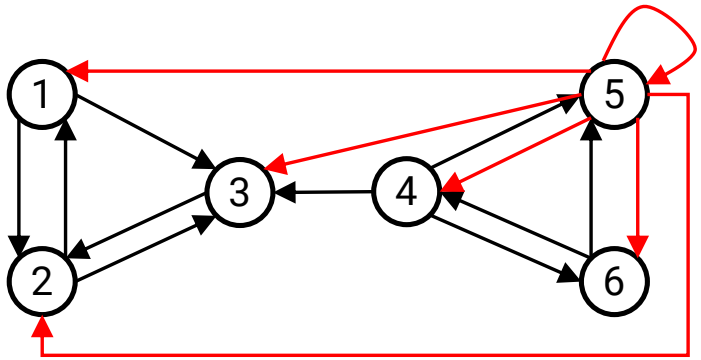


$$A = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

- Solution:
  - Remove dangling pages including links. Calculate PageRank and then include dangling pages back. Calculate their PageRank.
  - For each dangling page add out links to all pages in the Web.

Irreducible matrix

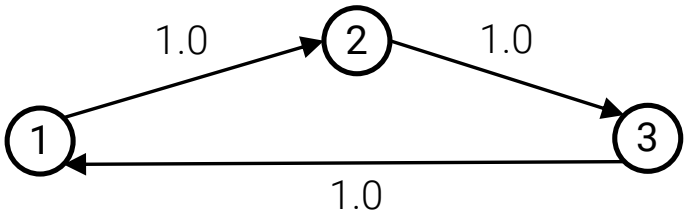
- If we make A stochastic, it can still remain irreducible. There is still no possibility to get from 3 to 4.



$$\overline{A} = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

Aperiodic matrix

- Aperiodic matrix has no periodic states. Web subgraph below is periodic. All states are periodic.



$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Making matrix irreducible and aperiodic

- Solution:
  - Add a link from each page to every page and give each link a small transition probability controlled by a parameter d.
- New rules for random surfer:
  - with probability d, randomly choose an out-link to follow.
  - with probability $1 - d$, jump to a random page without a link.

$$\boldsymbol{P} = \left((1-d)\frac{\boldsymbol{E}}{n} + d\boldsymbol{A}^T \right) \boldsymbol{P}$$

  - Where $\boldsymbol{e}$ is vector of all 1 and $E = \boldsymbol{e}\boldsymbol{e}^T$ is a matrix of $n \times n$ of all 1.

PageRank formula

- If we scale P so that $\boldsymbol{e}^T \boldsymbol{P} = n$ we get:

$$\boldsymbol{P} = (1-d)\boldsymbol{e} + d\boldsymbol{A}^T\boldsymbol{P}$$

- Which for page i is:

$$P(i) = (1-d) + d\sum_{j=1}^{n} A_{ji} P\,(\mathrm{j})$$

$$P(i) = (1-d) + d\sum_{(j,i)\in E}^{n} \frac{P(j)}{O_j}$$

PageRank algorithm

- Algorithm

```
PageRank-Iterate(G)
    P_0 ← e/n
    k ← 1
    repeat
        P_k ← (1 − d)e + dA^T P_{k−1}
        k ← k + 1
    until ||P_k − P_{k−1}|| < ε
    return P_k
```

PageRank strengths and weaknesses

- Strengths
  - query independent
  - hard to cheat
- Weaknesses
  - authority in general and not in connection with the query
  - time not considered